

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_6e519d4b-9e6\agent-aa3b6ae7d8d2dafbe.jsonl`
- SHA-256: `5961ec01e42a6540aefeb42311e141868436082922a11e7efa6c79bf54fdf2fa`
- Source modified: `2026-06-22T14:45:29+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_6e519d4b-9e6`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T14:44:22.144Z)

Read C:/Users/avidu/Projects/bhi-cloudrun/backend/worker/__main__.py (especially cmd_infer + the get_compute_backend usage around lines 160-220). Map EXACTLY how the CLI `backend.worker infer` dispatches a job to Cloud Run: how it builds the ComputeJobSpec, how it calls the compute backend's dispatch/poll, what it reads/writes to the bucket (--video-uri in, --out-uri out ? what files: intervals.csv? report.json? a done-marker?), and what the RESULT contains (does it return rally segments in-memory, or only write them to the bucket?). Quote the key lines with file:line. This is the path we must MIRROR for the API.

2. assistant (2026-06-22T14:44:31.232Z)

I'll read the file and map out the Cloud Run dispatch path.

3. user (2026-06-22T14:44:31.761Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import csv
24     import json
25     import logging
26     import os
27     import re
28     import sys
29     import tempfile
30     from typing import Any, List
```

```

27
28 from backend import storage
29 from backend.compute import ComputeJobSpec, get_compute_backend
30 from backend.config import (
31     StartupCheckError,
32     enforce_startup_checks,
33     load_config,
34     probe_cuda_available,
35 )
36 from backend.infrastructure.database import Database
37 from backend.pipeline.segmenters.base import segmenter_registry
38 from backend.pipeline.validators.base import registry as validator_registry
39 from backend.reporting import emit_indexer_report
40 from backend.results import Failure
41 from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
42 from backend.utils.hashing import compute_video_id
43
44 logger = logging.getLogger("backend.worker")
45
46
47 def _coerce(v: str) -> Any:
48     """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
49     low = v.lower()
50     if low in ("true", "false"):
51         return low == "true"
52     for cast in (int, float):
53         try:
54             return cast(v)
55         except ValueError:
56             pass
57     return v
58
59
60 def _apply_overrides(config: Any, sets: List[str]) -> None:
61     """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
62     for kv in sets or []:
63         key, sep, val = kv.partition("=")
64         if not sep:
65             raise ValueError(f"--set expects k=v, got {kv!r}")
66         parts = key.split(".")
67         obj = config
68         for p in parts[:-1]:
69             obj = getattr(obj, p)
70         setattr(obj, parts[-1], _coerce(val))
71
72
73 def _write_intervals_csv(segments: List[dict], path: str) -> None:
74     """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
75     artifact (same schema as the rally-annotator labels)."""
76     with open(path, "w", newline="") as f:
77         w = csv.writer(f)
78         w.writerow(["start", "end", "shot_count", "ending_reason"])
79         for s in segments:
80             w.writerow([
81                 f'{float(s.get("start_time", 0.0)):.3f}',
82                 f'{float(s.get("end_time", 0.0)):.3f}',
83                 s.get("shot_count", ""),
84                 s.get("ending_reason", s.get("shot_count_confidence", "")),
85             ])
86
87
88 def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->

```

```

None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),
95                             "end": float(s.get("end_time", 0.0))} for s in segments],
96             }, f, indent=2)
97
98
99     def _run_startup_checks(config: Any) -> bool:
100         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
101         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
102         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
103         (returns True, ZERO work) when no deployment.profile is selected.
104
105         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
106         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
107         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
108         no-op of work, not just of output."""
109         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
110         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
111         try:
112             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
113             return True
114         except StartupCheckError:
115             return False # already logged at ERROR by enforce_startup_checks
116
117
118     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
119         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
120         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
121         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
122
123         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
124         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
125         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
126         from backend.infrastructure.execution_policy import PolicyTimeout
127
128         spec = ComputeJobSpec(
129             video_uri=args.video_uri, out_uri=args.out_uri,
130             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
131         )
132         try:
133             result = compute.run_infer(spec)
134         except PolicyTimeout as e:
135             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
136             return 4
137         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
138                     result.video_id, result.returncode, result.outputs_uri)

```

```

139         return result.returncode
140
141
142     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
143                             int,
144                             status: str, storage_cfg: dict, scratch: str) -> None:
145         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
146         for a remote
147         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
148         return."""
149         try:
150             marker = {"video_id": video_id, "returncode": returncode,
151                       "segment_count": segment_count, "status": status}
152             local = os.path.join(scratch, "_done.json")
153             with open(local, "w", encoding="utf-8") as f:
154                 json.dump(marker, f)
155                 storage.put(local, done_uri, config=storage_cfg)
156                 logger.info("[worker] wrote completion marker -> %s", done_uri)
157             except Exception as e: # never fail a good run because the marker push hiccuped
158                 logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
159
160
161     def cmd_infer(args: argparse.Namespace) -> int:
162         config = load_config(args.config) if args.config else load_config()
163         _apply_overrides(config, args.set)
164         if not _run_startup_checks(config):
165             return 2
166
167         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
168         # process.
169         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
170         # through to the
171         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
172         # (the
173         # dispatcher forces compute_target=local_gpu), so it never recursively re-
174         # dispatches.
175         compute = get_compute_backend(config)
176         if compute is not None:
177             return _dispatch_remote(compute, args)
178
179         storage_cfg = config.storage.model_dump()
180
181         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
182         os.makedirs(scratch, exist_ok=True)
183         config.output.output_dir = scratch
184
185         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
186         # to scratch.
187         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
188                               config=storage_cfg)
189         print(f"[worker] pulled {args.video_uri} -> {local_video}")
190
191         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
192         # byte-identical).
193         video_id = compute_video_id(local_video)
194
195         # 3. VALIDATE (same registry as the main CLI).
196         val_results, val_context = validator_registry.run_all_with_context(local_video,
197                               config)
198         failures = [r for r in val_results if not r.passed]
199         db = Database(os.path.join(scratch, config.output.db_name))
200         db.add_video(video_id, local_video, "failed" if failures else "processed",
201                     [r.model_dump() for r in val_results])
202
203         segments: List[dict] = []

```

```

193     segmenter_actual = None
194     segmentation_ran = False
195     status = "failed"
196     try:
197         if failures:
198             print("[worker] validation FAILED: "
199                   + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
200             return 2
201         seg_name = args.segmenter or config.indexing.default_segmenter
202         segmenter_cls = segmenter_registry.get(seg_name)
203         if not segmenter_cls:
204             print(f"[worker] unknown segmenter: {seg_name!r} "
205                   f"(have {list(segmenter_registry.list_available())})")
206             return 2
207         segmenter_actual = seg_name
208         result = segmenter_cls(db, config).process_video(video_id, local_video,
209 max_frames=args.max_frames)
210         segmentation_ran = True
211         if isinstance(result, Failure):
212             print(f"[worker] segmenter FAILED: {result.message}")
213             return 3
214         segments = result.value if hasattr(result, "value") else result
215         status = "success"
216         # Loud, never-silent diagnostic: a 0-segment success is almost always a
217         # empty match ? the cold-start failure mode (the substrate detector yielded no
218         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
219         # for more).
220         if not segments:
221             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
222                             or getattr(config.indexing, "detector_impl", None) or
223 "auto")
224             logger.warning(
225                 "infer produced 0 segments for video_id=%s (segmenter=%s,
226 detector_impl=%s). "
227                 "The detector substrate yielded no usable trajectories/windows ? check
228 the detector "
229                 "logs above; common causes: native runner UNAVAILABLE
230 (weights/repo/WASB_PYTHON env), "
231                 "the WASB env produced no detections, or the input resolution differs
232 from the tuned "
233                 "proxy resolution. Re-run with -v for the native command + frame
234 counts.",
235                 video_id, segmenter_actual, detector_impl,
236             )
237         finally:
238             # Best-effort per-video observability report (never raises) ? parity with the
239             # main CLI.
240             emit_indexer_report(
241                 db=db, video_id=video_id, video_path=local_video, config=config,
242                 val_results=val_results, val_context=val_context,
243                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
244                 was_reingest=False, segmentation_ran=segmentation_ran,
245             )
246         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
247         out_local = os.path.join(scratch, "outputs", video_id)
248         os.makedirs(out_local, exist_ok=True)
249         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
250         _write_report_json(video_id, segments, status, os.path.join(out_local,
251 "report.json"))
252         out_prefix = args.out_uri.rstrip("/")
253         pushed = []

```

```

246     for fn in sorted(os.listdir(out_local)):
247         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
248         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
249         pushed.append(uri)
250
251     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
252     for u in pushed:
253         print("    ", u)
254
255     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
256     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
257     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
258     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
259     if getattr(args, "done_uri", None):
260         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
261     return 0
262
263
264     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
265     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
266     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
267
268     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
DD>_<name>.rallies.csv`,
269     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
270     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
271     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
272
273
274     def _label_clip_name(fname: str) -> str:
275         """Derive the clip name (the `videos/<name>.<ext>` join key) from a label
filename.
276
277         Strips both the `.rallies.csv` / `.csv` suffix and an optional leading ISO-date
prefix, so a
278         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
279
280         * `2026-06-22_GX020094.rallies.csv` -> `GX020094` (canonical, date-
stamped)
281         * `GX020094.rallies.csv` -> `GX020094` (legacy bare ? same
stem)
282         * `GX020094_proxy.rallies.csv` -> `GX020094_proxy` (`_proxy` is part of
the name, kept)
283         * `2026-06-21_mahadevpura_1.rallies.csv` -> `mahadevpura_1` (underscores in
<name> survive)
284
285         Without the date strip, `--trajectory-source detector` can't match a dated label
to its bare
286         video stem, so every clip is skipped and train aborts with `<2 videos`
(DATA_IN_GCS §7b #1).
287         """
288         name = fname.replace(".rallies.csv", "").replace(".csv", "")
289         return _LABEL_DATE_PREFIX.sub("", name)
290
291
292     def _probe_video_fps_width(path: str):

```

```

293     """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
294
295     The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
296     fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
297     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
298     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
299     probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
300     rather than guesses.
301     """
302     import json
303     import subprocess
304     try:
305         out = subprocess.check_output(
306             [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
307              "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
308             stderr=subprocess.DEVNULL).decode()
309         st = (json.loads(out).get("streams") or [{}])[0]
310         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
311         fps = float(num) / float(den or "1")
312         width = float(st.get("width") or 0)
313         if fps > 0 and width > 0:
314             return fps, width
315     except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
316         pass
317     return None
318
319
320 def _resolve_train_detector(args: argparse.Namespace, config: Any):
321     """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
322     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
323     stub=CPU mock). Returns the runner, or prints an error + returns None.
324
325     Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
326     detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
327     no shuttle detector and is rejected.
328     """
329     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
330
331     seg_cls = segmenter_registry.get(args.segmenter)
332     if not seg_cls:
333         print(f"[worker] unknown segmenter: {args.segmenter!r} "
334               f"(have {list(segmenter_registry.list_available())})")
335         return None
336     seg = seg_cls(None, config)
337     if not isinstance(seg, TrajectoryHybridSegmenter):
338         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
339               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
340     return None
341     try:
342         runner, _ = seg._resolve_runner(config.get("indexing", {}))
343     except NotImplementedError as e:
344         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
345         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
346         print(f"[worker] detector not available: {e}")

```

```

347         return None
348     ok, msg = runner.healthcheck()
349     if not ok:
350         print(f"[worker] detector environment not ready: {msg}")
351         return None
352     print(f"[worker] detector ready ({args.segmenter}): {msg}")
353     return runner
354
355
356 def cmd_train(args: argparse.Namespace) -> int:
357     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
358     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
359
360     Two trajectory sources (the train pipeline + harness are identical either way):
361     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
362         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
363     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
364         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
365         is the M3.5 "first real training" path; only the trajectory source flips.
366     """
367     import subprocess
368
369     config = load_config(args.config) if args.config else load_config()
370     _apply_overrides(config, args.set)
371     if not _run_startup_checks(config):
372         return 2
373     storage_cfg = config.storage.model_dump()
374
375     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
376     labels_dir = os.path.join(scratch, "labels")
377     traj_dir = os.path.join(scratch, "trajectories")
378     videos_dir = os.path.join(scratch, "videos")
379     os.makedirs(labels_dir, exist_ok=True)
380     os.makedirs(traj_dir, exist_ok=True)
381
382     corpus = args.corpus_uri.rstrip("/")
383     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
384                         if u.lower().endswith(".csv"))
385     if len(label_uris) < 2:
386         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
387             f"under {corpus}/labels/")
388         return 2
389
390     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
391     runner = None
392     video_by_stem: dict = {}
393     if args.trajectory_source == "detector":
394         os.makedirs(videos_dir, exist_ok=True)
395         runner = _resolve_train_detector(args, config)
396         if runner is None:
397             return 2
398         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
399             vname = storage.parse_uri(vuri).name
400             if not vname.lower().endswith(_VIDEO_EXTS):
401                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
402                 video_by_stem[os.path.splitext(vname)[0]] = vuri

```

```

403
404     print(f"[worker] trajectory source: {args.trajectory_source}")
405     specs: List[dict] = []
406     for uri in label_uris:
407         fname = storage.parse_uri(uri).name # e.g.
2026-06-22_GX020094.rallies.csv
408         name = _label_clip_name(fname) # -> GX020094 (date
prefix stripped)
409         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
410         if args.trajectory_source == "detector":
411             vuri = video_by_stem.get(name)
412             if not vuri:
413                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
414                 continue
415                 local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
416                                     config=storage_cfg)
417                 video_id = compute_video_id(local_video)
418                 # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
419                 # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
420                 meta = _probe_video_fps_width(local_video)
421                 if meta is None:
422                     print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
423                     continue
424                     fps, frame_width = meta
425                     traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
426                     if not traj:
427                         print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
428                         continue
429                     specs.append({"name": name, "trajectory": traj, "labels": local_label,
430                                "fps": fps, "frame_width": frame_width})
431         else:
432             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
433             traj = os.path.join(traj_dir, f"{name}_traj.csv")
434             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
435             specs.append({"name": name, "trajectory": traj, "labels": local_label,
436                           "fps": args.fps, "frame_width": args.frame_width})
437
438     if len(specs) < 2:
439         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
440         return 2
441     manifest_path = os.path.join(scratch, "manifest.json")
442     with open(manifest_path, "w", encoding="utf-8") as f:
443         json.dump(specs, f, indent=2)
444     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
445     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
446
447     out_dir = os.path.join(scratch, "artifacts")
448     cmd = [sys.executable, "-m", "training.gen0.harness",
449            "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
450     if args.floor:
451         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
452                        if "://" in args.floor else args.floor)
453         cmd += ["--floor", floor_local]

```

```

454     print("[worker] training:", " ".join(cmd))
455     rc = subprocess.run(cmd).returncode
456     if rc != 0:
457         print(f"[worker] gen0 harness failed (rc={rc})")
458         return rc
459
460     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
461     bundle = os.path.join(out_dir, args.version)
462     out_prefix = args.out_uri.rstrip("/")
463     pushed = []
464     for fn in sorted(os.listdir(bundle)):
465         uri = f"{out_prefix}/weights/{args.version}/{fn}"
466         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
467         pushed.append(uri)
468     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
469     for u in pushed:
470         print("    ", u)
471     return 0
472
473
474     def build_parser() -> argparse.ArgumentParser:
475         p = argparse.ArgumentParser(prog="python -m backend.worker",
476                                     description="Storage-aware worker: pull ? run pipeline ?
477                                     push.")
478         p.add_argument("-v", "--verbose", action="store_true",
479                       help="DEBUG logging (the native detector command, frame counts, per-
480                       stage detail)")
481         sub = p.add_subparsers(dest="cmd", required=True)
482
483         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
484         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
485         gcs://")
486         pi.add_argument("--out-uri", required=True, help="output prefix
487         (outputs/<video_id>/? is appended)")
488         pi.add_argument("--segmenter", default=None, help="default:
489         indexing.default_segmenter")
490         pi.add_argument("--config", default=None, help="config.json path (default:
491         ./config.json)")
492         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
493         dir)")
494         pi.add_argument("--max-frames", type=int, default=None)
495         pi.add_argument("--done-uri", default=None,
496                       help="M6: storage URI for a completion MARKER (video_id + rc)
497                       written on a "
498                       "successful run, so a remote (Cloud Run) dispatcher's bucket-
499                       poll terminates. "
500                       "Default-OFF: omit it for the normal local run.")
501         pi.add_argument("--set", action="append", default=[], metavar="k=v",
502                       help="config override, e.g. --set indexing.skip_ai_handoff=true")
503         pi.set_defaults(func=cmd_infer)
504
505         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
506         trajectories -> harness)")
507         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
508         videos/ for "
509         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
510         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
511         is appended)")
512         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
513         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
514                       default="synth",
515                       help="synth = CPU-mock label-consistent trajectories (default, no
516                       GPU); "
517                       "detector = REAL shuttle trajectories via the resolved
518                       DetectorRunner ")

```

```

504             "(needs videos/ in the corpus; honours detector_impl).")
505         pt.add_argument("--segmenter", default="wasb_hybrid",
506             help="trajectory hybrid whose detector is used for --trajectory-
source detector")
507         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
508         pt.add_argument("--config", default=None)
509         pt.add_argument("--scratch", default=None)
510         pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
511         pt.add_argument("--frame-width", type=float, default=1280.0,
512             help="synth trajectory frame width (detector probes the video)")
513         pt.add_argument("--set", action="append", default=[], metavar="k=v")
514         pt.set_defaults(func=cmd_train)
515         return p
516
517
518     def _setup_logging(verbose: bool) -> None:
519         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
520         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
521         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
522         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
523         entry), not at import, so importing the module never reconfigures a caller's
logging."""
524         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
525         # configured logging at import), which would leave the level at WARNING and re-
suppress the
526         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
527         logging.basicConfig(
528             level=logging.DEBUG if verbose else logging.INFO,
529             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
530             datefmt="%H:%M:%S",
531             force=True,
532         )
533
534
535     def main(argv=None) -> int:
536         args = build_parser().parse_args(argv)
537         _setup_logging(getattr(args, "verbose", False))
538         return args.func(args)
539
540
541     if __name__ == "__main__":
542         sys.exit(main())
543

```

4. assistant (2026-06-22T14:44:35.924Z)

This is a read-only mapping task. Let me check the related modules to fully ground the dispatch path, particularly the compute backend's `run\_infer` and `ComputeJobSpec`.

5. user (2026-06-22T14:44:36.329Z)

```

C:/Users/avidu/Projects/bhi-cloudrun/backend/compute\base.py
C:/Users/avidu/Projects/bhi-cloudrun/backend/compute\__init__.py
C:/Users/avidu/Projects/bhi-cloudrun/backend/compute\cloud_run.py

```

6. user (2026-06-22T14:44:38.708Z)

```

1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:

```

```
53         """Run ``spec`` to completion and return its ``ComputeResult``."""
54
```

7. user (2026-06-22T14:44:39.185Z)

```
1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5     ``None``
6     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
7     behaviour, byte
8     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
9     ``cpu_mock`` nothing
10    changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
11    needed).
12    """
13
14    from __future__ import annotations
15
16    import os
17    from typing import Any, Optional
18
19    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
20    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
21
22    __all__ = [
23        "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
24    ]
25
26    def _storage_dict(config: Any) -> dict:
27        storage = getattr(config, "storage", None)
28        if storage is None:
29            return {}
30        if hasattr(storage, "model_dump"):
31            return storage.model_dump() # type: ignore[no-any-return]
32        return dict(storage) if isinstance(storage, dict) else {}
33
34    def get_compute_backend(
35        config: Any,
36        *,
37        run_client: Optional[Any] = None,
38        policy: ExecutionPolicy = DEFAULT_POLICY,
39        token: Optional[str] = None,
40    ) -> Optional[ComputeBackend]:
41        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
42        to run
43        in-process (the default-OFF path).
44
45        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
46        ``CloudRunCompute``). Its
47        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
48        ``CLOUD_RUN_REGION``
49        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
50        injectable (tests
51        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
52        verified on M7).
53        """
54        target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
55        if target != "cloud_run":
56            return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
57
58        # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
```

```

selected.
52     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57     client = run_client if run_client is not None else GoogleCloudRunJobClient()
58     return CloudRunCompute(
59         run_client=client, job_name=job_name, region=region, project=project,
60         storage_config=_storage_dict(config), policy=policy, token=token,
61     )
62

```

8. user (2026-06-22T14:44:39.722Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19    owner-verified on the M7 real run.
20    """
21
22    from __future__ import annotations
23
24    import json
25    import logging
26    import os
27    import tempfile
28    import uuid
29    from abc import ABC, abstractmethod
30    from typing import Any, List, Optional
31
32    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34    from backend.storage import fetch, parse_uri
35    from backend.storage import get_storage
36
37    LOG = logging.getLogger("compute.cloud_run")
38
39    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch

```

```

branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                     project: Optional[str] = None, storage_config: Optional[dict] = None,
87                     policy: ExecutionPolicy = DEFAULT_POLICY,
88                     token: Optional[str] = None,
89                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name

```

```

92         self._region = region
93         self._project = project
94         self._storage_config = storage_config or {}
95         self._policy = policy
96         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98         self._token = token
99         self._container_overrides = (
100             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101         )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
project=self._project)
115         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
label="cloud-run-infer", logger=LOG)
118         video_id = str(marker.get("video_id", ""))
119         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
120         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
121         rc = int(marker.get("returncode", 1))
122         if not video_id:
123             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
124                         "video_id) ? treating as failure", self._job_name)
125             rc = rc or 1
126         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
127         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
128         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
129         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
130
131     # --- storage-backed completion signal (bucket-poll) ---
132     def _read_marker(self, done_uri: str) -> Optional[dict]:
133         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
134         ref = parse_uri(done_uri)
135         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
136             return None
137         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
138
139     def _read_json(self, uri: str) -> dict:
140         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
141         # raises ValueError ? unlike local's identity passthrough. Without an explicit

```

```

dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

9. assistant (2026-06-22T14:44:43.101Z)

I now have the complete picture. Let me also confirm the execution policy's `poll\_until` / `PolicyTimeout` semantics briefly since they govern the poll budget.

10. user (2026-06-22T14:44:45.073Z)

```

4:- ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) + retry-
with-backoff
39:class PolicyTimeout(TimeoutError):
40:    """An op hung past ``op_timeout_sec`` or never became ready within ``max_wait_sec``."""
44:class ExecutionPolicy:
48:    op_timeout_sec: float = 120.0          # HARD per-attempt deadline ? the hang-killer that
was missing
52:    on_timeout: str = "reschedule"        # reschedule | partial | fail (consumed by P2/P3)
70:DEFAULT_POLICY = ExecutionPolicy()
81:    """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
98:        except BaseException as e:      # relay ANY error (incl. timeouts) to the
caller thread
104:        if done.wait(timeout=policy.op_timeout_sec):
113:            last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
115:            policy.op_timeout_sec)
125:def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy = DEFAULT_POLICY,
*,

```

11. user (2026-06-22T14:44:48.115Z)

```

44    class ExecutionPolicy:
45        """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46        mode: WaitMode = WaitMode.WAIT_AND_RESUME
47        poll_every_n_sec: float = 10.0
48        op_timeout_sec: float = 120.0      # HARD per-attempt deadline ? the hang-killer
that was missing
49        max_wait_sec: float = 1800.0      # total wall budget for a poll loop
50        max_retries: int = 5               # retries (=> max_retries+1 attempts) for
failures/hangs
51        backoff_base_sec: float = 3.0     # exponential: backoff_base * 2**attempt
52        on_timeout: str = "reschedule"    # reschedule | partial | fail (consumed by
P2/P3)
53        resumable: bool = True
54
55        def to_dict(self) -> dict:
56            d = asdict(self)
57            d["mode"] = self.mode.value
58            return d

```

```

59
60     @classmethod
61     def from_dict(cls, d: dict) -> "ExecutionPolicy":
62         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63         kw = {k: v for k, v in d.items() if k in known}
64         if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65             kw["mode"] = WaitMode(kw["mode"])
66         return cls(**kw)
67
68
69     #: The sane default (also what a job gets if it declares no policy).
70     DEFAULT_POLICY = ExecutionPolicy()
71
72
73     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:

```

12. assistant (2026-06-22T14:45:29.840Z)

I have the full path mapped. Here is the exact dispatch flow.

How `backend.worker.infer` dispatches a job to Cloud Run

Entry: `cmd\_infer` decides local-vs-cloud via the compute factory

`backend/worker/\_\_main\_\_.py:158` `cmd\_infer`. After config load + startup checks, the branch point is:

```

- `__main__.py:168` ? `compute = get_compute_backend(config)`
- `__main__.py:169-170` ? `if compute is not None: return _dispatch_remote(compute, args)`

```

The factory (`backend/compute/\_\_init\_\_.py:47-48`) returns non-`None` **only** when `config.deployment.compute\_target == "cloud\_run";` `""` / `local\_gpu` / `cpu\_mock` ? `None` ? fall through to the inline pull?segment?push path (the rest of `cmd\_infer`). So cloud dispatch is gated purely on that config field. Cloud Run coordinates come from env, not config (`\_\_init\_\_.py:54-56`): `CLOUD\_RUN\_JOB` (default `rally-worker`), `CLOUD\_RUN\_REGION` (default `asia-south1`), `GOOGLE\_CLOUD\_PROJECT`.

Building the `ComputeJobSpec`

`\_dispatch\_remote` (`\_\_main\_\_.py:118-139`) constructs it directly from the CLI args:

```

...
__main__.py:128-131
spec = ComputeJobSpec(
    video_uri=args.video_uri, out_uri=args.out_uri,
    segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
)
result = compute.run_infer(spec) # __main__.py:133
...

```

`ComputeJobSpec` (`backend/compute/base.py:20-30`) is a frozen dataclass: `video\_uri`, `out\_uri`, `segmenter`, `config\_overrides: tuple[str,...]`. The frozen/tuple choice is deliberate (hashable spec).

Dispatch + poll ? `CloudRunCompute.run\_infer`

`backend/compute/cloud\_run.py:103-132`. This is the heart of the mirror target:

1. **Derives a private done-marker URI** keyed by a per-call uuid token so concurrent jobs to one bucket don't collide:

```

- `cloud_run.py:104-106` ? `out_root = spec.out_uri.rstrip("/")`; `token = self._token or uuid.uuid4().hex`; `done_uri = f"{out_root}/{_dispatch}/{token}.done"`

```

2. **Builds the container argv** ? literally the same ``backend.worker infer` CLI`, with ``--done-uri`` added:

```
- `cloud_run.py:107-112`:
    ...
    argv = ["infer", "--video-uri", spec.video_uri, "--out-uri", spec.out_uri, "--done-uri",
done_uri]
    if spec.segmenter: argv += ["--segmenter", spec.segmenter]
    for kv in (*spec.config_overrides, *self._container_overrides): argv += ["--set", kv]
    ...

- `self._container_overrides` defaults to `_DEFAULT_CONTAINER_OVERRIDES`
(`cloud_run.py:41-44`): `deployment.compute_target=local_gpu` and
`indexing.detector_impl=native`. These force the container to run inline on the L4 GPU and
disarm the dispatch guard so it never recursively re-dispatches.


3. Fires the Cloud Run Job execution (non-blocking):



```
- `cloud_run.py:114-115` ? `execution = self._client.run_job(self._job_name, argv,
region=..., project=...)`. The real client (`GoogleCloudRunJobClient.run_job`,
`cloud_run.py:64-79`) lazy-imports `google.cloud.run_v2`, overrides the job's container `args`
with this `argv`, and starts it. It does not wait.

4. Bucket-polls the done-marker (NOT the Cloud Run API status ? D16):


```
- `cloud_run.py:119-120` ? `marker = poll_until(lambda: self._read_marker(done_uri),
self._policy, ...)`. `_read_marker` (`cloud_run.py:135-140`) returns `None` (keep polling) until
the object `exists()` in storage, then reads it as JSON. `poll_until` runs under
`DEFAULT_POLICY`: `poll_every_n_sec=10`, `max_wait_sec=1800` total wall budget,
`op_timeout_sec=120` per attempt (`execution_policy.py:47-49`). Exhausting the budget raises
`PolicyTimeout`.
```


```


```

What is read/written in the bucket

Container (the worker running inline on GPU) WRITES ? ``cmd_infer`` inline path,

```
`__main__.py:238-261`:
- `__main__.py:239-242` ? serializes two files into `scratch/outputs/<video_id>/`:
    - `**`intervals.csv`** via `_write_intervals_csv` (`__main__.py:73-85`): header
`start,end,shot_count,ending_reason`, decimal-second `[start,end)` rallies ? the join-friendly
artifact matching the rally-annotator label schema.
    - `**`report.json`** via `_write_report_json` (`__main__.py:88-96`): `{video_id, status,
segment_count, segments:[{start,end}]}`.
- `__main__.py:244-249` ? pushes every file under `out_local` to
`{out_uri}/outputs/<video_id>/<fn>` via `storage.put`. Output path is keyed on the recomputed
MD5 `video_id` (`__main__.py:183`), so it's deterministic regardless of where the job ran.
- `__main__.py:259-260` ? because `--done-uri` was passed, writes the completion marker via
`_write_done_marker` (`__main__.py:142-155`): a tiny JSON `{video_id, returncode, segment_count,
status}` put to `done_uri`. Written on SUCCESS only (`__main__.py:258` comment) ? a
failed/hung container writes no marker, so the dispatcher's poll times out (failure-marker is a
noted follow-up).
```

Dispatcher READS ? ``CloudRunCompute.run_infer``, ``cloud_run.py:121-132``:

```
- From the marker: `video_id = marker.get("video_id", "")` and `rc = int(marker.get("returncode",
1))`. A missing returncode defaults to 1 (failure), never 0 (`cloud_run.py:123-124`); an
empty/unreadable marker (no `video_id`) is forced to failure (`cloud_run.py:125-128`).
- `cloud_run.py:129-130` ? derives `outputs_uri = f"{out_root}/outputs/{video_id}/"` and reads
report.json from there into a dict: `report =
self._read_json(f"{outputs_uri}report.json")`.
```

What the RESULT contains ? and the key mirroring caveat

```
`run_infer` returns a `ComputeResult` (`base.py:33-43`):
...
cloud_run.py:132
return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id, report=report)
...

Fields: `returncode`, `outputs_uri`, `video_id`, `report` (the parsed `report.json` dict).
```

The rally segments are NOT returned in-memory across the cloud boundary. The dispatcher
never sees the segment list as Python objects ? it only:

```
- gets back the summary `report.json` (which has `segment_count` and a list of `{start,
```

end}` only ? no `shot_count`/`ending_reason`), and
- knows `outputs\_uri`, the bucket prefix where the **full** `intervals.csv` + `report.json` **durably** live.

So to get the actual rally intervals, **the caller must read**
`{out\_uri}/outputs/{video\_id}/intervals.csv` (and/or `report.json`) back from the bucket itself. **`\_dispatch\_remote`** (`\_\_main\_\_.py:137-139`) just logs `video\_id`, `returncode`, `outputs\_uri` and returns `result.returncode` as the worker exit code. `\_dispatch\_remote` also maps `PolicyTimeout` ? a clean **rc 4** (`\_\_main\_\_.py:134-136`), distinct from local rc 2 (validation/config) / rc 3 (segmenter failure).

Summary for the API mirror

To mirror this path, the API must:

1. Set `deployment.compute\_target=cloud\_run` (and have
`CLOUD\_RUN\_JOB`/`CLOUD\_RUN\_REGION`/`GOOGLE\_CLOUD\_PROJECT` in env), then build a
`ComputeJobSpec(video_uri=<gs://in>, out_uri=<gs://out-root>, segmenter=...,
config_overrides=...)` and call `get_compute_backend(config).run_infer(spec)` ? or replicate
`run\_infer`'s dispatch+poll directly.
2. Treat the call as **async-but-blocking**: it dispatches a Cloud Run Job then bucket-polls a
`{out\_uri}/dispatch/<token>.done` marker until done or `PolicyTimeout` (default 1800s wall
budget). For a web API you will likely want to split dispatch from poll rather than block a
request thread on `poll\_until`.
3. On completion, take `result.video\_id` + `result.outputs\_uri` and **fetch** `intervals.csv` from
the bucket **to get the full rally segments** (start/end/shot_count/ending_reason) ?
`result.report` only carries the coarse `{start,end}` summary, and nothing returns segments in-
memory.

Relevant files (absolute):

- `C:/Users/avidu/Projects/bhi-cloudrun/backend/worker/\_\_main\_\_.py` (`cmd\_infer` :158,
`\_dispatch\_remote` :118, writers :73/:88, done-marker :142, marker write :259)
- `C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/\_\_init\_\_.py` (factory gate :47)
- `C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/base.py` (`ComputeJobSpec` :20,
`ComputeResult` :33)
- `C:/Users/avidu/Projects/bhi-cloudrun/backend/compute/cloud\_run.py` (`run\_infer` :103, argv
build :107, poll :119, result :132, marker read :135)
- `C:/Users/avidu/Projects/bhi-cloudrun/backend/infrastructure/execution\_policy.py`
(`poll\_until`, `PolicyTimeout` :39, `ExecutionPolicy` defaults :44-53)